

Competitive Programming

Section 43 Editorial: String Algorithms I

Competitive Programming Course

2026-07-06

Table of contents

1	Editorial overview	3
1.1	Detailed solution guide	3
2	A. KMP Occurrences	4
2.1	Restatement	4
2.2	Observations and algorithm	4
2.3	Correctness proof	4
2.4	Complexity	4
2.5	Full C++ solution	4
2.6	Full Python solution	4
2.7	Tests that matter	5
3	B. All Borders	6
3.1	Restatement	6
3.2	Observations and algorithm	6
3.3	Correctness proof	6
3.4	Complexity	6
3.5	Full C++ solution	6
3.6	Full Python solution	6
3.7	Tests that matter	7
4	C. Minimal Period	8
4.1	Restatement	8
4.2	Observations and algorithm	8
4.3	Correctness proof	8
4.4	Complexity	8
4.5	Full C++ solution	8
4.6	Full Python solution	8
4.7	Tests that matter	9
5	What to carry forward	10

1 Editorial overview

1.1 Detailed solution guide

A. KMP Occurrences. Build the combined string `pattern + '#' + text`, where `#` is a separator that cannot appear in either input. The prefix function value `pi[i]` is the length of the longest prefix of the combined string ending at `i`. When `pi[i] == m`, an occurrence of the pattern has just ended. The end is at combined index `i`, the text starts at combined index `m + 1`, so the 1-based start position in the text is `i - 2*m + 1`. Overlapping matches are handled automatically because the prefix function falls back through borders instead of resetting to zero.

B. All Borders. A border of `s` is both a prefix and a suffix. The longest proper border has length `pi[n-1]`. If a string of length `k` is a border, then its own longest proper border has length `pi[k-1]`, and that is also a border of the original string. Repeating this prefix-link walk lists every border from longest to shortest; reversing gives increasing order. No border is skipped, because every shorter border of `s` must be a border of the longest border that contains it.

C. Minimal Period. If the longest border has length `b`, then the first candidate period is `p = n - b`: the suffix that repeats the prefix begins after `p` characters. This candidate is valid only when `n % p == 0`; otherwise the repeated blocks do not tile the string and the answer is `n`. This is why strings such as `ababab` return 2, while `ababa` returns 5.

Common pitfalls. The separator must be absent from the alphabet, positions for A are 1-based, and B should not print the full length because a border is proper in this exercise.

The prefix function is the only real tool in this section. It is computed once in linear time and then interpreted in three ways.

- A builds `pattern + separator + text`; whenever `pi[i]` equals the pattern length, an occurrence ends at `i`.
- B follows prefix links from `pi[n-1]` to list every proper border of the full string.
- C uses the longest border to derive the candidate period `n - pi[n-1]`, then checks divisibility.

The important proof idea is that prefix links enumerate exactly the shorter borders, so no possible border or period is skipped.

2 A. KMP Occurrences

2.1 Restatement

Print the number and 1-based positions of all occurrences of pattern in text.

2.2 Observations and algorithm

Use the detailed solution guide above as the roadmap. The implementation below follows the stated invariant directly and handles the edge cases called out for this exercise.

2.3 Correctness proof

Correctness follows from the problem-specific argument in the detailed guide: the maintained structure represents exactly the objects claimed there, and each answer is read from that representation. $\square$

2.4 Complexity

The running time is the one derived in the detailed guide and visible in the loops below: pre-processing builds the structure once, while each operation performs the stated scans, searches, jumps, or graph traversals.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; vector<int> pi(string s){vector<int> p(s.size());for(int i=1;i<(int)s.size();i++)p[i]=0;for(int j=1;j<(int)s.size();j++){int k=pi[s[j]-1];while(k>0&& s[j]!=s[k])k=pi[k-1];p[j]=k+1;}return p;}
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string pat,text;if(!(cin>>pat>>text))return 0;vector<int> p=pi(pat);for(int i=0;i<(int)text.size();i++){int k=0;while(k<p.size()&& text[i-k]!=pat[k])k++;if(k==p.size())cout<<i+1<<" ";}
```

2.6 Full Python solution

```

import sys
def pi(s):
    p=[0]*len(s)
    for i in range(1,len(s)):
        j=p[i-1]
        while j and s[i]!=s[j]: j=p[j-1]
        if s[i]==s[j]: j+=1
        p[i]=j
    return p
def main():
    lines=sys.stdin.read().splitlines(); pat=lines[0]; text=lines[1]; s=pat+"#"+text; p=pi(s)
    for i,v in enumerate(p):
        if v==len(pat): ans.append(i-2*len(pat)+1)
    print(len(ans)); print(" ".join(map(str,ans)) if ans else "")
if __name__=="__main__": main()

```

2.7 Tests that matter

Use singleton inputs, repeated values, empty answers, ancestor/self paths, disconnected matching edges, and bottleneck flow graphs.

3 B. All Borders

3.1 Restatement

Print all proper border lengths of a string in increasing order.

3.2 Observations and algorithm

Use the detailed solution guide above as the roadmap. The implementation below follows the stated invariant directly and handles the edge cases called out for this exercise.

3.3 Correctness proof

Correctness follows from the problem-specific argument in the detailed guide: the maintained structure represents exactly the objects claimed there, and each answer is read from that representation. $\square$

3.4 Complexity

The running time is the one derived in the detailed guide and visible in the loops below: pre-processing builds the structure once, while each operation performs the stated scans, searches, jumps, or graph traversals.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;if(!(c
```

3.6 Full Python solution

```
import sys
def pi(s):
    p=[0]*len(s)
    for i in range(1,len(s)):
        j=p[i-1]
        while j and s[i]!=s[j]: j=p[j-1]
        p[i]=j
```

```

        if s[i]==s[j]: j+=1
        p[i]=j
    return p
def main():
    s=sys.stdin.readline().strip(); p=pi(s); ans=[]; k=p[-1] if s else 0
    while k: ans.append(k); k=p[k-1]
    print(" ".join(map(str,ans[::-1])))
if __name__=="__main__": main()

```

3.7 Tests that matter

Use singleton inputs, repeated values, empty answers, ancestor/self paths, disconnected matching edges, and bottleneck flow graphs.

4 C. Minimal Period

4.1 Restatement

Print the shortest period length if the string is repetitions of a block, otherwise **n**.

4.2 Observations and algorithm

Use the detailed solution guide above as the roadmap. The implementation below follows the stated invariant directly and handles the edge cases called out for this exercise.

4.3 Correctness proof

Correctness follows from the problem-specific argument in the detailed guide: the maintained structure represents exactly the objects claimed there, and each answer is read from that representation. $\square$

4.4 Complexity

The running time is the one derived in the detailed guide and visible in the loops below: pre-processing builds the structure once, while each operation performs the stated scans, searches, jumps, or graph traversals.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);string s;if(!(c
```

4.6 Full Python solution

```
import sys
def pi(s):
    p=[0]*len(s)
    for i in range(1,len(s)):
        j=p[i-1]
        while j and s[i]!=s[j]: j=p[j-1]
        p[i]=j
```

```

        if s[i]==s[j]: j+=1
        p[i]=j
    return p
def main():
    s=sys.stdin.readline().strip(); n=len(s); k=n-pi(s)[-1]
    print(k if n%k==0 else n)
if __name__=="__main__": main()

```

4.7 Tests that matter

Use singleton inputs, repeated values, empty answers, ancestor/self paths, disconnected matching edges, and bottleneck flow graphs.

5 What to carry forward

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.